



# *INDIA.RUNS*

Build what next India runs on

Team Name : Redrob Semantic Ranker

Team Leader Name : Nimisha Agarwal

Problem Statement : Intelligent Candidate Discovery & Ranking

## Solution Overview

- **Two-part score:** a semantic match on each candidate's career story (45%) blended with a structured fit rubric (55%), then adjusted by real-world reality checks.
- **Reads the work, not the tags:** we score the prose in summaries and job descriptions, not the skills keyword list.
- **Reality-check multipliers:** disqualifier rules, behavioral availability, location, experience band and a honeypot sink scale each score up or down.
- **What sets it apart:** it reasons about the gap between what a JD says and what it means — surfacing plain-language fits and rejecting keyword-stuffers a traditional matcher ranks highly.
- **Production-ready:** standard-library Python, CPU-only, no network, ~2 min for 100K candidates.

## JD Understanding & Candidate Evaluation

- **Key JD requirements:** production retrieval / ranking / recommendation & vector search; rigorous evaluation (NDCG, MRR); real product (not pure-research/consulting) background; 5–9 yrs as a range; India Tier-1 or willing to relocate.
- **Explicit do-NOT-want:** pure researchers, consulting-only careers, CV/speech-only, recent-LangChain-only, non-coding seniors, and job-hoppers.
- **Most important signals:** the career-history descriptions (what they actually built), behavioral availability, and profile consistency.
- **Beyond keywords:** evidence of building search/ranking/recsys in the prose outweighs the skills tags; skills are trust-weighted so unused “expert” tags count as padding.

## Ranking Methodology

- **Retrieve & represent:** TF-IDF over career prose; cosine similarity to a JD concept-query (retrieval, ranking, embeddings, evaluation, production, NLP/IR).
- **Structured rubric (55%):** domain evidence 0.30, role-fit 0.26, production 0.14, evaluation 0.10, trust-weighted skills 0.10, experience 0.10.
- **Final score:**  $(0.45 \cdot \text{semantic} + 0.55 \cdot \text{rubric}) \times \text{disqualifier-gates} \times \text{behavioral} \times \text{location} \times \text{honeypot} \times \text{experience}$ .
- **Heuristics over hosted LLMs:** transparent, deterministic scoring that fits the CPU / no-network budget; two streaming passes keep memory under 1 GB.

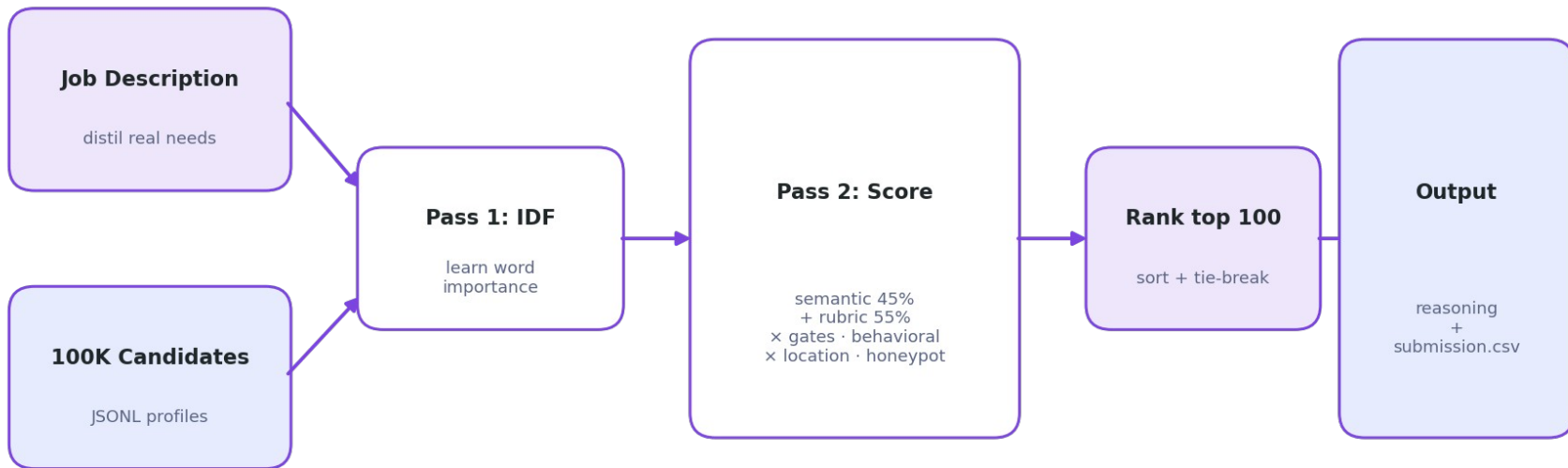
## Explainability & Data Validation

- **Every rank explained:** a 1-2 sentence reason built only from that candidate's facts – title, years, company, career evidence, and an honest concern where one exists.
- **No hallucination:** reasons reference only fields present in the profile, and the tone always matches the rank.
- **Suspicious profiles (honeypots):** caught by internal contradictions – tenure exceeding experience, dates vs. duration mismatches, a single role longer than the whole career, unused “expert” skills – then multiplied by 0.02 to sink them.
- **Validated result:** 0 honeypots and 0 non-engineering decoys in the top 100.

## End-to-End Workflow

- 1 · Read JD: distil real requirements into a weighted concept-query plus a rule set.
- 2 · Stream candidates (pass 1): learn corpus word-importance (IDF) over all profiles.
- 3 · Score (pass 2): semantic match + rubric + multipliers + honeypot check, per candidate.
- 4 · Rank: sort by score and keep the top 100 with a deterministic tie-break.
- 5 · Explain: generate a specific, fact-based reason for each candidate.
- 6 · Output: a spec-compliant submission.csv (candidate\_id, rank, score, reasoning).

## System Architecture



CPU-only · no network · standard-library Python · ~2 min for 100K · < 1 GB RAM

## Results & Performance

- **Right people at the top:** Senior/Staff ML, NLP, Recommendation-Systems and Search Engineers at product companies (Zomato, Razorpay, Paytm, CRED, Nykaa, Sarvam AI).
- **Trap resistance:** 0 honeypots and 0 non-engineering titles in the top 100; 88 of 100 inside the 5–9-year band.
- **Availability matters:** strong-but-unreachable profiles (e.g. Meta / Microsoft engineers with ~7–11% recruiter-response) correctly fall below the cutoff.
- **Meets the constraints:** CPU-only, no network or GPU, ~2 min for 100K, < 1 GB RAM, single command; passes the official format validator.

## Technologies Used

- Python 3 standard library only: json, csv, re, math, heapq, collections, datetime – chosen for zero-dependency, fully-reproducible, no-network ranking.
- Hand-implemented TF-IDF: a lightweight semantic match with no heavyweight model downloads, fitting the 5-minute CPU budget.
- Rule / heuristic scoring: transparent, debuggable and defensible at interview, unlike opaque per-candidate LLM calls.
- matplotlib & python-pptx: for the charts and this deck – build-time only, never in the ranking path.
- Git / GitHub: versioned, reproducible submission.

## Submission Assets

- GitHub repo: [github.com/nimishaagarwal20/redrob-semantic-ranker](https://github.com/nimishaagarwal20/redrob-semantic-ranker) — full code, README, one-command reproduce.
- Ranked output: `submission.csv` — the top 100 with scores and reasoning (format-validated).
- Approach deck: `approach_deck.pdf` (this deck).
- Runnable sandbox: a self-contained Google Colab notebook ( $\leq 100$ -candidate demo).
- Reproduce: `python rank.py -candidates ./candidates.jsonl -out ./submission.csv`



# *INDIA.RUNS*

Build what next India runs on

# THANK YOU